SEMANA 4

ACTIVIDAD 1 Visualización de Lista Enlazada

- Documento con los diagramas generados, anotados con tus propias palabras explicando: (1) por qué insertar al inicio es O(1), (2) por qué insertar al final es O(n), y (3) qué problema surge al eliminar el primer nodo que no ocurre con otros nodos.

Lista inicial

Código
cabeza —▶ [7|●]—▶[15|●]—▶[23|●]—▶[42|●]—▶ null

1. Insertar al inicio (valor 3)

ANTES

Código
cabeza —▶ [7|●]—▶[15|●]—▶[23|●]—▶[42|●]—▶ null

PASOS

1. Crear nuevo nodo:

   Código

   nuevo —▶ [3|●]—▶ null

2. Apuntar nuevo al antiguo cabeza:

   Código

   [3|●]—▶[7|●]—▶[15|●]—▶[23|●]—▶[42|●]—▶ null

3. Actualizar cabeza:

   Código

   cabeza —▶ [3|●]—▶[7|●]—▶[15|●]—▶[23|●]—▶[42|●]—▶ null

DESPUÉS

Código
cabeza —▶ [3|●]—▶[7|●]—▶[15|●]—▶[23|●]—▶[42|●]—▶ null

Anotación: Insertar al inicio es **O(1)** porque solo se crean un nodo y se actualizan dos punteros. No importa cuántos elementos tenga la lista.

2. Insertar al final (valor 50)

ANTES

Código
cabeza —► [3|●]—►[7|●]—►[15|●]—►[23|●]—►[42|●]—► null

PASOS

1. Crear nuevo nodo:

   Código

   nuevo —► [50|●]—► null

2. Recorrer hasta el último nodo:

   Código

   actual —► [42|●]—► null

3. Apuntar último nodo al nuevo:

   Código

   [42|●]—►[50|●]—► null

DESPUÉS

Código
cabeza —► [3|●]—►[7|●]—►[15|●]—►[23|●]—►[42|●]—►[50|●]—► null

Anotación: Insertar al final es **O(n)** porque hay que recorrer toda la lista hasta encontrar el último nodo. El costo depende del tamaño de la lista.

3. Eliminar nodo con valor 15

ANTES

Código
cabeza —► [3|●]—►[7|●]—►[15|●]—►[23|●]—►[42|●]—►[50|●]—► null

PASOS

1. Recorrer hasta encontrar 15:

   Código

anterior —► [7|●]—►[15|●]...
actual —► [15|●]—►[23|●]...

2. Saltar el nodo 15:

   Código

   [7|●]—►[23|●]—►[42|●]—►[50|●]—► null

DESPUÉS

Código
cabeza —► [3|●]—►[7|●]—►[23|●]—►[42|●]—►[50|●]—► null

Anotación: Eliminar un nodo intermedio es **O(n)** porque hay que recorrer la lista hasta encontrarlo. La eliminación en sí es O(1), pero la búsqueda cuesta.

Caso especial: Eliminar el primer nodo

ANTES

Código
cabeza —► [3|●]—►[7|●]—►[23|●]—►[42|●]—►[50|●]—► null

PASO

- Actualizar directamente cabeza:

  Código

  cabeza —► [7|●]—►[23|●]—►[42|●]—►[50|●]—► null

DESPUÉS

Código
cabeza —► [7|●]—►[23|●]—►[42|●]—►[50|●]—► null

Anotación: El problema especial al eliminar el primer nodo es que no existe un "anterior" que apunte al siguiente. Se debe actualizar directamente la referencia cabeza. Esto lo hace distinto de eliminar cualquier otro nodo.

Conclusiones

1. **Insertar al inicio es O(1):** porque solo se manipulan punteros locales sin recorrer la lista.
2. **Insertar al final es O(n):** porque hay que recorrer toda la lista para encontrar el último nodo.
3. **Eliminar el primer nodo es un caso especial:** no hay nodo anterior, por lo que se debe actualizar directamente la referencia cabeza.

ACTIVIDAD 2 Implementación Guiada de Lista Enlazada

- Código fuente completo de tu implementación con: (1) Clase Nodo, (2) Clase Lista Enlazada con al menos 6 métodos funcionales, (3) Programa de prueba que demuestre cada operación, (4) Comentarios explicando la lógica de cada método.

```
#include <iostream>

using namespace std;



class Nodo {

public:

   int dato;

   Nodo* siguiente;

   Nodo(int valor) {

      dato = valor;

      siguiente = nullptr;

   }

};

class ListaEnlazada {

private:

   Nodo* cabeza;

   int tam;

public:

   ListaEnlazada() {

      cabeza = nullptr;

      tam = 0;

   }
```

```
bool estaVacia() {
    return cabeza == nullptr;
}
void insertarInicio(int valor) {
    Nodo* nuevo = new Nodo(valor);
    nuevo->siguiente = cabeza;
    cabeza = nuevo;
    tam++;
}
void insertarFinal(int valor) {
    Nodo* nuevo = new Nodo(valor);
    if (estaVacia()) {
        cabeza = nuevo;
    } else {
        Nodo* temp = cabeza;
        while (temp->siguiente != nullptr) {
            temp = temp->siguiente;
        }
        temp->siguiente = nuevo;
    }
    tam++;
}
void eliminarInicio() {
    if (estaVacia()) {
```

```
        cout << "La lista está vacía, no se puede eliminar." << endl;
        return;
    }
    Nodo* temp = cabeza;
    cabeza = cabeza->siguiente;
    delete temp;
    tam--;
}
bool buscar(int valor) {
    Nodo* temp = cabeza;
    while (temp != nullptr) {
        if (temp->dato == valor) {
            return true;
        }
        temp = temp->siguiente;
    }
    return false;
}
void imprimir() {
    if (estaVacia()) {
        cout << "La lista está vacía." << endl;
        return;
    }
    Nodo* temp = cabeza;
```

```
        cout << "Lista: ";
        while (temp != nullptr) {
            cout << temp->dato << " -> ";
            temp = temp->siguiente;
        }
        cout << "NULL" << endl;
    }
    int obtenerTam() {
        return tam;
    }
};
int main() {
    ListaEnlazada lista;
    cout << "¿Lista vacía?: " << (lista.estaVacia() ? "Sí" : "No") << endl;
    lista.insertarInicio(10);
    lista.insertarInicio(20);
    lista.imprimir();
    lista.insertarFinal(30);
    lista.insertarFinal(40);
    lista.imprimir();
    cout << "Buscar 30: " << (lista.buscar(30) ? "Encontrado" : "No encontrado") << endl;
    cout << "Buscar 50: " << (lista.buscar(50) ? "Encontrado" : "No encontrado") << endl;
    lista.eliminarInicio();
    lista.imprimir();
```

```
    cout << "Tamaño de la lista: " << lista.obtenerTam() << endl;

    return 0;}
```

- estaVacia() → Comprueba si cabeza == nullptr.
- insertarInicio(valor) → Crea un nodo y lo pone como nuevo primero.
- insertarFinal(valor) → Recorre hasta el último nodo y lo conecta al nuevo.
- eliminarInicio() → Avanza la cabeza y libera el nodo anterior.
- buscar(valor) → Recorre la lista comparando valores.
- imprimir() → Recorre la lista mostrando cada nodo.
- obtenerTam() → Devuelve el contador de elementos.

ACTIVIDAD 3 Resolver Problemas Clásicos con Depuración Visual

- Implementación de al menos 2 problemas clásicos (invertir lista + encontrar medio) con: (1) Código funcional probado, (2) Documento explicando el algoritmo paso a paso en tus propias palabras, (3) Análisis de complejidad temporal y espacial.

```
#include <iostream>

using namespace std;

class Nodo {

public:

    int dato;

    Nodo* siguiente;

    Nodo(int valor) {

        dato = valor;

        siguiente = nullptr;

    }

};

class ListaEnlazada {

private:

    Nodo* cabeza;
```

```
public:
    ListaEnlazada() { cabeza = nullptr; }
    void insertarFinal(int valor) {
        Nodo* nuevo = new Nodo(valor);
        if (!cabeza) {
            cabeza = nuevo;
        } else {
            Nodo* temp = cabeza;
            while (temp->siguiente) temp = temp->siguiente;
            temp->siguiente = nuevo;
        }
    }
    void imprimir() {
        Nodo* temp = cabeza;
        while (temp) {
            cout << "[" << temp->dato << "] -> ";
            temp = temp->siguiente;
        }
        cout << "NULL" << endl;
    }
    void invertir() {
        Nodo* anterior = nullptr;
        Nodo* actual = cabeza;
        Nodo* siguiente = nullptr;
```

```
        while (actual != nullptr) {
            siguiente = actual->siguiente;
            actual->siguiente = anterior;
            anterior = actual;              /
            actual = siguiente;
        }
        cabeza = anterior;
    }
    Nodo* encontrarMedio() {
        if (!cabeza) return nullptr;
        Nodo* lento = cabeza;
        Nodo* rapido = cabeza;
        while (rapido != nullptr && rapido->siguiente != nullptr) {
            lento = lento->siguiente;
            rapido = rapido->siguiente->siguiente;
        }
        return lento;
    }
};
int main() {
    ListaEnlazada lista;
    lista.insertarFinal(1);
    lista.insertarFinal(2);
    lista.insertarFinal(3);
```

```
    lista.insertarFinal(4);

    lista.insertarFinal(5);

    cout << "Lista original: ";

    lista.imprimir();

    lista.invertir();

    cout << "Lista invertida: ";

    lista.imprimir();

    Nodo* medio = lista.encontrarMedio();

    if (medio) {

        cout << "Nodo medio: " << medio->dato << endl;

    } else {

        cout << "La lista está vacía." << endl;

    }

    return 0;

}
```

Problema 1: Invertir lista

Idea:
Usamos tres punteros (anterior, actual, siguiente).

- Guardamos el siguiente nodo.
- Invertimos el puntero del nodo actual para que apunte hacia atrás.
- Avanzamos anterior y actual.
- Al final, anterior será la nueva cabeza.

Ejemplo:

Original: 1 → 2 → 3 → 4 → 5 → NULL Invertida: 5 → 4 → 3 → 2 → 1 → NULL

Complejidad:

- Tiempo: O(n) (recorremos todos los nodos una vez).

- Espacio: O(1) (solo usamos tres punteros).

Problema 2: Encontrar el nodo medio

Idea:
Usamos dos punteros:

- lento avanza de uno en uno.
- rápido avanza de dos en dos.
  Cuando rápido llega al final, lento está en el medio.

Ejemplo:

Lista: 1 → 2 → 3 → 4 → 5 → NULL Medio: 3

Complejidad:

- Tiempo: O(n) (recorremos la lista una vez).
- Espacio: O(1) (solo dos punteros).

Conclusión

Hemos implementado dos algoritmos clásicos:

1. Invertir lista enlazada
2. Encontrar el nodo medio

ACTIVIDAD 4 Análisis de Casos de Uso Reales

- Documento comparativo que incluya: (1) Tu análisis inicial de cada escenario ANTES de consultar la IA, (2) El análisis de la IA, (3) Tu reflexión sobre las diferencias entre ambos análisis, (4) Conclusiones sobre cuándo usar cada estructura.

(1) Tu análisis inicial (hipótesis antes de consultar la IA)

- **A) Playlist de música:**
  Pensé en una **lista enlazada doble** porque permite insertar y eliminar en cualquier posición sin mover muchos elementos, y navegar hacia adelante/atrás.
- **B) Buffer de teclado:**
  Supuse que una **lista enlazada simple** sería buena porque los caracteres llegan uno a uno y se procesan en orden.
- **C) Sistema de versiones de documento:**
  Imaginé un **arreglo dinámico** porque es fácil acceder a la última versión y recorrerlo hacia atrás con índices.

(2) Análisis de la IA

- **A) Playlist de música:**
  Recomienda **arreglo dinámico (vector)** porque el acceso por índice es O(1), lo cual es más importante que optimizar inserciones.
- **B) Buffer de teclado:**
  Recomienda **cola circular con arreglo** porque el tamaño es fijo y pequeño, y la latencia mínima es crítica.
- **C) Sistema de versiones de documento:**
  Recomienda **lista doblemente enlazada** porque se accede mucho a la última versión y se navega hacia atrás, sin necesidad de acceso aleatorio.

(3) Reflexión sobre diferencias

- **Playlist:** Yo pensé en lista doble, pero la IA me mostró que el acceso por índice es más frecuente que las inserciones. En este caso, el arreglo dinámico es mejor porque el tamaño es moderado (50–500 canciones).
- **Buffer de teclado:** Yo pensé en lista enlazada, pero la IA me corrigió: los punteros generan overhead y no son cache-friendly. El arreglo circular es óptimo porque el tamaño es fijo y pequeño.
- **Versiones de documento:** Yo pensé en arreglo, pero la IA me mostró que las inserciones al final y la navegación hacia atrás son más frecuentes que el acceso aleatorio. La lista doblemente enlazada es más natural para este caso.

(4) Conclusiones sobre cuándo usar cada estructura

- **Arreglo dinámico (vector):**
  Úsalo cuando el acceso aleatorio es frecuente y el tamaño es moderado. Ejemplo: playlist de música.
- **Lista enlazada simple:**
  Úsala cuando solo insertas/eliminás en extremos y no necesitas acceso aleatorio. Ejemplo: pila o cola básica.
- **Lista doblemente enlazada:**
  Úsala cuando necesitas navegar hacia adelante y atrás, y las inserciones/eliminaciones en medio son frecuentes. Ejemplo: historial de versiones.
- **Cola circular con arreglo:**
  Úsala para buffers pequeños y de tiempo real, donde la latencia mínima es crítica. Ejemplo: buffer de teclado.

Escenario	Tu hipótesis inicial	Arreglo dinámico	Acceso por índice más importante	Conclusión
Playlist	Lista doble	Arreglo dinámico	Acceso por índice más importante	Vector es mejor
Buffer teclado	Lista simple	Cola circular con arreglo	Latencia y cache	Cola circular
Versiones documento	Arreglo dinámico	Lista doble	Navegación atrás más frecuente	Lista doble

ACTIVIDAD 5 Validación de Implementación con Técnica de "Rubber Duck"

- Transcripción de la sesión de "rubber duck debugging" con al menos 3 métodos diferentes. Para cada método, incluye: (1) El código original, (2) Las preguntas de la IA y tus respuestas, (3) Bugs o mejoras que descubriste durante la explicación, (4) Código corregido si aplica.

Método 1: insertarFinal

(1) Código

```
void insertarFinal(int valor) { Nodo* nuevo = new Nodo(valor);

 if (cabeza == nullptr) { cabeza = nuevo;

 } else { Nodo* temp = cabeza;

 while (temp->siguiente != nullptr) { temp = temp->siguiente;

} temp->siguiente = nuevo;

} tamaño++;

}
```

(2) Preguntas del pato y respuestas

- Pato: ¿Puedes explicarme qué hace la primera línea de tu código y por qué es necesaria?
- Tú: Creo un nuevo nodo con el valor que quiero insertar. Es necesario porque sin él no tendría qué añadir a la lista.
- Pato: ¿Qué pasa si la lista está vacía?
- Tú: Si cabeza es nullptr, asigno el nuevo nodo como cabeza.
- Pato: ¿Cómo sabes que llegaste al último nodo?
- Tú: Porque verifico que temp->siguiente sea nullptr.
- Pato: ¿Qué pasaría si olvidas actualizar el tamaño?
- Tú: La lista crecería pero la variable tamaño quedaría desfasada.

(3) Bugs o mejoras detectadas

- No manejo el caso de memoria insuficiente al crear el nodo.
- El método funciona, pero podría ser más eficiente si mantuviera un puntero al último nodo (cola).

(4) Código corregido

```
void insertarFinal(int valor) { Nodo* nuevo = new Nodo(valor);

 if (!nuevo) return;

 if (cabeza == nullptr) { cabeza = nuevo;

 } else { Nodo* temp = cabeza; while (temp->siguiente != nullptr) { temp = temp->siguiente;

 } temp->siguiente = nuevo;

} tamaño++; }
```

Método 2: eliminarInicio

(1) Código

```
void eliminarInicio() { if (cabeza == nullptr) return;

Nodo* temp = cabeza;

 cabeza = cabeza->siguiente;

 delete temp; tamaño--; }
```

(2) Preguntas del pato y respuestas

- Pato: ¿Qué pasa si la lista está vacía?
- Tú: Retorno inmediatamente, no hay nada que eliminar.
- Pato: ¿Qué ocurre con el nodo eliminado?
- Tú: Lo borro con delete para liberar memoria.
- Pato: ¿Qué pasaría si olvidas decrementar el tamaño?
- Tú: La lista se reduciría pero la variable tamaño quedaría incorrecta.

(3) Bugs o mejoras detectadas

- No manejo el caso de lista de un solo elemento explícitamente, aunque el código sí lo cubre.
- Podría ser útil devolver el valor eliminado.

(4) Código corregido

```
int eliminarInicio() { if (cabeza == nullptr) return -1;

Nodo* temp = cabeza;

int valor = temp->dato;

cabeza = cabeza->siguiente;

delete temp;

 tamaño--;

return valor;

}
```

Método 3: buscar

(1) Código

```
bool buscar(int valor) { Nodo* temp = cabeza;

 while (temp != nullptr) { if (temp->dato == valor) return true;

 temp = temp->siguiente;

 }

 return false;

 }
```

(2) Preguntas del pato y respuestas

- Pato: ¿Qué pasa si la lista está vacía?
- Tú: El while no se ejecuta y retorna false.
- Pato: ¿Cómo sabes que encontraste el valor?
- Tú: Comparo temp->dato con el valor buscado.
- Pato: ¿Qué pasaría si hay valores repetidos?
- Tú: Retorno true en la primera coincidencia, no sigo buscando.

(3) Bugs o mejoras detectadas

- No devuelve la posición del valor encontrado, solo un booleano.
- Podría ser útil devolver un puntero al nodo.

(4) Código corregido

```
Nodo* buscar(int valor)

 {

Nodo* temp = cabeza;

while (temp != nullptr) { if (temp->dato == valor) return temp;

temp = temp->siguiente;

 }

return nullptr;

 }
```

Conclusiones

Descubriste que tu código funciona, pero puede mejorar en robustez (manejo de memoria, valores de retorno).

- El “pato de hule” te ayudó a detectar huecos como: qué pasa si la lista está vacía, si hay repetidos, o si olvidas actualizar el tamaño.
- La depuración línea por línea te obligó a justificar cada instrucción, lo que fortaleció tu comprensión.

ACTIVIDAD 6 Diseño de Sistema: Lista con Acceso O(1) al Final

- Implementación mejorada de ListaEnlazada con referencia a cola que incluya: (1) Diagrama de la nueva estructura, (2) Código modificado con todos los métodos actualizados para mantener 'cola' consistente, (3) Análisis de cómo cambió la complejidad de cada operación, (4) Pruebas que demuestren que insertarFinal() ahora es O(1).

Diagrama de la nueva estructura

ASCII para visualizarlo:

```
cabeza ---> [dato|sig] ---> [dato|sig] ---> [dato|NULL] <--- cola
```

- cabeza apunta al primer nodo.
- cola apunta directamente al último nodo.
- Cuando la lista está vacía: cabeza = NULL, cola = NULL.

(2) Código modificado en C++

```
#include using namespace std;

 struct Nodo { int dato; Nodo* sig; Nodo(int d) : dato(d), sig(nullptr) {} };

 class ListaEnlazada { private: Nodo* cabeza;

Nodo* cola;

 public: ListaEnlazada() : cabeza(nullptr), cola(nullptr) {} bool estaVacia() { return cabeza == nullptr;
} void insertarInicio(int valor) { Nodo* nuevo = new Nodo(valor);

 if (estaVacia()) { cabeza = cola = nuevo;

 } else { nuevo->sig = cabeza; cabeza = nuevo;

} } void insertarFinal(int valor) { Nodo* nuevo = new Nodo(valor);

 if (estaVacia()) { cabeza = cola = nuevo;

 } else { cola->sig = nuevo; cola = nuevo;

 } } void eliminarInicio() { if (estaVacia()) return; Nodo* temp = cabeza;

cabeza = cabeza->sig; if (cabeza == nullptr) cola = nullptr;

 } void eliminarFinal() { if (estaVacia()) return;

 if (cabeza == cola) { // solo un nodo delete cabeza;

 cabeza = cola = nullptr; return;

} Nodo* actual = cabeza; while (actual->sig != cola) { actual = actual->sig; } delete cola;

cola = actual;

 cola->sig = nullptr; } void mostrar() { Nodo* actual = cabeza;

while (actual ¡= nullptr) { cout << actual->dato << " -> ";

 actual = actual->sig; } cout << "NULL\n"; } };
```

(3) Análisis de complejidad

Operación	Antes (solo cabeza)	Ahora (cabeza + cola)
insertarInicio	O(1)	O(1)
insertarFinal	O(n)	O(1)
eliminarInicio	O(1)	O(1)
eliminarFinal	O(n)	O(n) (no mejora, porque necesitamos recorrer hasta el penúltimo nodo)
Mostrar()	O(n)	O(n)

Pruebas

```
int main() { ListaEnlazada lista;

lista.insertarFinal(10);

 lista.insertarFinal(20);

 lista.insertarFinal(30);

lista.mostrar();

lista.insertarInicio(5);

 lista.mostrar();

 lista.eliminarInicio(); lista.mostrar();

lista.eliminarFinal();

 lista.mostrar();

return 0;

}
```

Resultado esperado:

10 -> 20 -> 30 -> NULL 5 -> 10 -> 20 -> 30 -> NULL 10 -> 20 -> 30 -> NULL 10 -> 20 -> NULL